

Adaptive Progressive Retrieval Attention: Factorized Control, Query Reinterpretation, and Bounded Retry

Jorge Simao

EInnovator R&D – Center for the Sciences of Computation, Cognition, and Complexity

August 21, 2026

Abstract

Progressive Retrieval Attention (PRA) separates conceptual memory discovery from physical native-key/value (K/V) activation. This paper asks how much PRA effort a query should receive and when a bounded second attempt is justified. We first evaluate three coherent effort profiles, then open their coupling into independently selectable interpretation, search, and admission controls.

The factorized study replays frozen native graph and query scores for 95 validation and 65 held-out HotpotQA/QASPER example-seed units. It evaluates 792 valid configurations per unit over query facets F , roots R , successor breadth K , hop depth H , conceptual budget B_s , and physical K/V admission budget B_{kv} . On held-out HotpotQA, the factorized oracle preserves complete-chain quality 1.000 while reducing mean abstract cost from 34.18 for the cheapest sufficient E0/E1/E2 profile to 19.98. On the 24 QASPER units solved by both action spaces, cost falls from 16.11 to 8.95; factorization additionally solves one unit that no profile solves, yielding .833 versus .800 overall. Thus profile coupling hides substantial lower-cost combinations, although no learned router yet predicts those combinations reliably.

Five-seed factorized controllers make that limitation explicit. R0 profile routing obtains .631 chain quality at cost 24.29. Independent, conservative, signed-hash query, and autoregressive heads reduce cost to 16.13–18.06 but quality to .486–.560 because under-allocation rises. Parameter accuracy is therefore not the endpoint. A one-retry evaluator-side correction audit raises quality from .631 to the factorized ceiling .923, correcting all 15 initial HotpotQA failures and four of nine QASPER failures at 2.49 mean added cost; eight HotpotQA and QASPER cases use a single root/facet change, while compound fallback remains necessary elsewhere. This is retry headroom, not a learned online policy.

Query interpretation is itself an action. Explicit and structural regions remove recent-head serialization sensitivity in matched controls and remain stable through 8K following tokens. Structural selection is perfect on six semi-natural fixtures but falls to .400 under five deliberate role decoys, while the competing recent-head interpretation remains correct there. The result motivates disagreement-aware reinterpretation rather than unconditional structural replacement. All primary quality results are frozen-score evidence-chain endpoints, not generated-answer scores. Kernel, cache, batching, and serving-engine optimization are handed to Papers 5.5 and 6.

1 Introduction

A fixed memory policy is wrong in two directions. A narrow policy wastes quality on questions whose evidence is dispersed or connected by several relations. A broad policy wastes search, transfer, K/V capacity, and attention competition on questions that can be answered from one compact

region. The relevant systems question is therefore not only which memory should be retrieved, but how much effort the current query justifies.

PRA provides several independent control surfaces. A runtime may form one or several query facets, retain one or several roots, expand different numbers of neighbors and hops, search at different layers, and materialize anything from a compact evidence interval to a broader source region. Paper 3 showed that conceptual breadth and physical disclosure are not the same budget: once identity is fixed, exact native-K/V evidence can outperform a whole parent while using far fewer physical states. The present work turns those controls into an adaptive inference contract.

The central test is

$$Q_{\text{adaptive}} \approx Q_{\text{always-high}} \quad \text{while} \quad \mathbb{E}[C_{\text{adaptive}}] < C_{\text{always-high}}, \quad (1)$$

where Q is held-out output or retrieval quality and C includes conceptual search, physical K/V, generation retries, and measured latency. Equation 1 is not guaranteed on every dataset. If almost every query is hard, the correct controller should spend the high budget.

This paper makes seven contributions:

1. It defines a public PRA control vector and freezes three monotonic effort profiles rather than predicting unconstrained continuous parameters.
2. It implements hand-rule and learned minimum-effort controllers using only observable query, routing, output, and memory features; evaluator labels are rejected by the serving API.
3. It provides manual and automatic retry modes with incremental state reuse, hard budgets, semantic answer consistency, and structured decision traces.
4. It evaluates validation-frozen adaptive compute, calibration, retry recovery, and oracle minimum effort on held-out HotpotQA and QASPER traces.
5. It makes query-region location and extent explicit, supports structured prompt roles, and tests matched query-first, query-last, query-middle, log, and URL-list layouts.
6. It measures profile quantization regret over 792 valid factorized configurations per unit, including token-weighted evidence precision and recall and separate search/admission budgets.
7. It retrains profile, independent-head, signed-hash query, conservative, and autoregressive routers on validation-derived factorized targets rather than inherited profile labels.
8. It evaluates bounded single-control correction and compound fallback as explicit retry upper bounds, recording wrong-to-correct, correct-to-broken, extra work, and action identity.

Full-weight PRA retraining is deliberately absent. That intervention belongs to Paper 4. Here the scientific object is adaptive inference over frozen routing and consumer behavior.

2 Adaptive PRA as a Control Problem

2.1 Control vector

At attempt t , the runtime selects

$$a_t = (\mathcal{Q}, F, R, K, H, B_c, B_{kv}, \theta, L_s, L_c, G, M), \quad (2)$$

where $\mathcal{Q}$ is one or more functional query regions, F is the facet policy, R retained roots, K candidate neighbors per expansion, H hop depth, B_c conceptual parent budget, B_{kv} physical native-K/V budget, θ a closure threshold, L_s search layers, L_c consumer layers, G routing/materialization granularity, and M disclosure policy. We separate B_c and B_{kv} because Paper 3 found that broad search can remain compatible with compact evidence-only activation.

The first public implementation uses three profiles (Table 1). They are not universal hyperparameters; they are a stable SDK contract and an initial experimental ladder.

Table 1: Validation-frozen effort profiles. The layer identifiers refer to the inherited 28-layer Qwen experiments. Lower routing thresholds permit broader closure.

Profile	$\mathcal{Q}$	facets	R	K	H	B_c	B_{kv}	G	consumer layers
E0 low	head	1	1	2	0	2	256	256	27
E1 medium	structural	2	2	4	1	4	512	128	26–27
E2 high	multi-region	4	4	8	3	8	1024	64	24–27

Escalation is monotonic in permitted facets, roots, neighbors, hops, conceptual budget, K/V budget, and layer sets. A lower θ is broader. Code rejects a ladder in which a later profile removes a prior resource. Monotonic permission does not imply monotonic answer quality: extra memory may dilute attention, so retries must also record correct-to-incorrect transitions.

2.2 Observable state and leakage boundary

The controller may observe cheap query statistics, score gaps, routing entropy, root competition, facet agreement, frontier shape, newly discovered memory, path convergence, output entropy, answer log probability and margin, retry consistency, active K/V, memory-attention ratio, and a relevance-density proxy. Ground-truth answers, oracle identities, gold evidence, and evidence recall are evaluator fields. They cannot enter the serving feature constructor.

For vocabulary distribution p_t , token entropy is

$$H_t = - \sum_v p_t(v) \log p_t(v), \quad \bar{H} = \frac{1}{T} \sum_{t=1}^T H_t. \quad (3)$$

Entropy is one signal. A fluent wrong answer can have low entropy. We therefore compare entropy-only, answer-margin, routing-only, and combined predictors rather than defining confidence as a single scalar by fiat.

2.3 Minimum-effort prediction

For validation example i , let

$$e_i^* = \min\{e \in \{E0, E1, E2\} : Q_i(e) \geq q_i^*\}, \quad (4)$$

where q_i^* is the target quality criterion. If no profile succeeds, $E2$ is retained as the bounded fallback. The oracle e_i^* is used only to train or analyze the controller. At test time, a standardized ridge classifier predicts one profile from the initial observable feature vector. The classifier is deliberately small, deterministic, and serializable without a new runtime dependency.

Controller regret is

$$\text{regret}_i = C(\hat{e}_i) - C(e_i^*) \quad (5)$$

subject to reporting whether $Q_i(\hat{e}_i)$ matches $Q_i(e_i^*)$. Negative regret without matched quality is not a saving.

2.4 Query-region discovery: separating intent from position

The runtime first asks which prompt region specifies the retrieval objective and only then asks how to facet that region. These are distinct decisions:

$$\text{prompt} \rightarrow \mathcal{Q} = \{[s_j, e_j]\}_j \rightarrow \{q_1, \dots, q_F\} \rightarrow \text{roots} \rightarrow \text{search}. \quad (6)$$

Functional roles include instruction, query, current state, context, references, history, and tool or log output. Position does not determine role. Explicit token spans or caller-labeled segments are authoritative. Raw text falls back to a deterministic structural ladder that recognizes question/instruction headings and punctuation while discounting code fences, logs, context labels, and URL lists. A retry may reinterpret a recent-head cue as a structural or multi-region cue before increasing R , K , H , or K/V budget.

The selection records start, end, region count, method, confidence, and score margin. Multiple regions support a persistent instruction plus a narrower current question; session mode selects the smallest current query state rather than the whole conversation. The heuristic is deliberately a baseline. A learned span scorer is justified only if explicit-span headroom remains after structural selection on natural prompts.

2.5 Learned effort routing: profiles to interacting heads

All controller families target minimum sufficient effort rather than the highest-scoring action:

$$a_i^* = \arg \min_a C(a) \quad \text{subject to} \quad Q_i(a) \geq q_i^*. \quad (7)$$

R0 selects one validation-frozen profile. R1 uses a shared feature MLP and independent categorical heads. R2 adds a cached query-region representation. R3A predicts controls autoregressively in the order $\mathcal{Q} \rightarrow F \rightarrow R \rightarrow K \rightarrow H \rightarrow B_s \rightarrow B_{kv}$. Integer controls and thresholds use finite categorical values; no Cartesian action class is constructed. The first comparison inherits profile-derived labels; the decisive comparison is retrained later on independently measured factorized-oracle targets. For independent heads,

$$\mathcal{L} = \sum_j \lambda_j \text{CE}(\hat{a}_j, a_j^*). \quad (8)$$

Evaluation reports each head, under- and over-allocation, quality regret, cost regret, calibration, and latency. Joint exact match is secondary to the held-out quality/expected-effort frontier.

3 Retry and Runtime Design

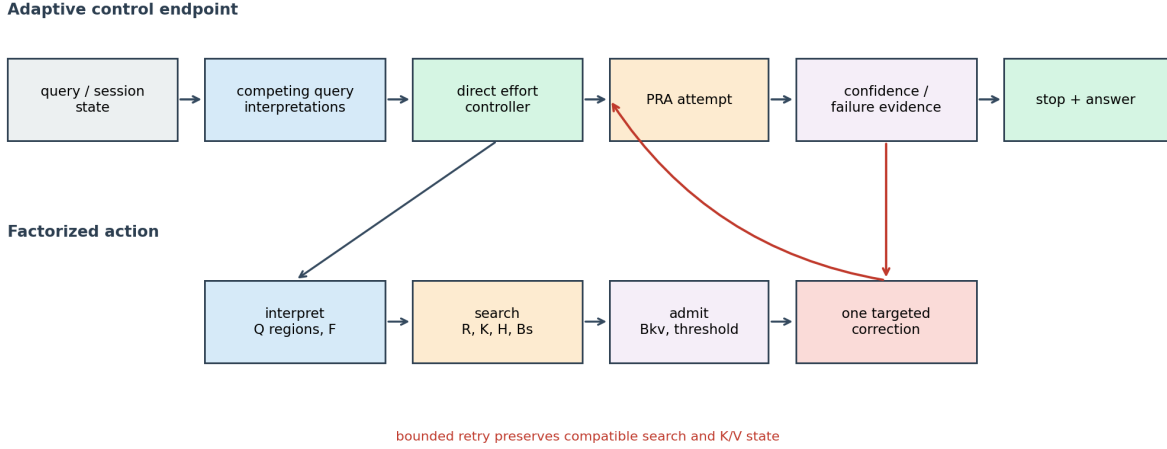


Figure 1: Adaptive PRA control endpoint. Competing query interpretations feed a direct profile or factorized action. Observable failure evidence may trigger one targeted correction with compatible search and K/V reuse; otherwise the controller stops.

3.1 State machine

Attempt zero uses the selected profile or factorized action. A manual caller may name low, medium, or high effort, provide individual controls, and request one bounded retry. Automatic mode applies a calibrated stop rule. If it fails, the corrective action may reinterpret the query, widen facets, increase roots, neighbors, hops, or search budget, or change K/V admission while preserving compatible roots, search results, K/V state, and prefix state. Search budget, active K/V, retry count, and latency are hard caps.

The public transition is:

```

profile = controller.choose(observable_features)
while True:
    result = execute(profile, reusable_state=previous)
    stop, signals = stop_policy(result.features, result.error_probability)
    trace.append(public_decision(result, profile, signals))
    if stop or budget_exhausted(profile, trace):
        return result, trace
    profile = targeted_correction(profile, observable_failure_signals)
    previous = result
  
```

Answers across attempts are compared with normalized token F1. This is a cheap semantic proxy, not a full entailment judge. The runtime records initially-wrong-to-correct, initially-correct-to-wrong, and no-change retries separately.

3.2 Structured traces

A trace contains attempt index, profile and control vector, active K/V, selected-parent count, incorrect-answer probability, routing entropy, answer margin, consistency, search/materialization/

generation time, reuse counts, query-region bounds, confidence, method and expansion count, escalation signals, and stop reason. It does not contain hidden chain-of-thought. Evaluator outcomes live in experiment rows outside the controller feature path.

3.3 Control/execution boundary

Let $Q \in \mathbb{R}^{n_q \times d}$ be native query states and $K_m \in \mathbb{R}^{n_m \times d}$ indexed memory keys. Exact batched search computes

$$S = QK_m^\top, \quad I = \text{TopK}(S, k). \quad (9)$$

This equation defines the search work exposed to the controller. How a later runtime compiles or indexes it is outside the adaptive-policy contribution; approximation is acceptable only under a separate matched-recall systems gate.

3.4 Search/admission boundary

Conceptual search returns source-parent identities. Admission maps a bounded subset to native K/V with shape $[B, H, T, D_h]$. The adaptive trace must record both the searched parent budget and the physically admitted token count. PagedAttention controls where K/V is stored [5]; PRA control decides which logical memory becomes active. Physical layout optimization is deferred.

4 Experimental Protocol

4.1 Adaptive-compute cohort

The controller study reuses frozen Paper-2.5 semantic exploration traces. The selected root policy is seed agreement at .6; transition policy is fixed Top-4. Fractions .1, .2, and .4 instantiate E0, E1, and E2. There are 95 validation and 65 held-out example-seed units across HotpotQA and QASPER. Validation chooses the hand rule, learned classifier, failure predictor, and retry threshold. No threshold is changed after held-out evaluation.

The quality endpoint is complete evidence-chain recovery. This is a routing endpoint, not generated answer accuracy. Output confidence is therefore evaluated separately with Paper 3’s controlled model, using its validation/held-out split and actual prediction entropy, NLL, answer margin, and correctness. Keeping the two cohorts separate avoids fabricating a joint output/routing signal that was not measured.

Abstract effort is deterministic:

$$C(e) = RK(H + 1) + B_c + B_{kv}/64. \quad (10)$$

A retry pays the final monotonic profile plus a fixed controlled regeneration charge of two units per additional attempt. Measured routing latency is reported separately.

4.2 Factorized-control cohort

The same 95/65 grouped split is replayed from frozen Qwen native adjacency, query-facet scores, source spans, and evidence groups. Validation supplies factorized router targets; held-out targets are evaluator-only. The full valid lattice is executed on both partitions so the held-out oracle is an evaluation ceiling, not a pruning input. Pareto dominance maximizes complete-chain quality, evidence recall, and evidence precision while minimizing abstract cost, active K/V, and transition comparisons.

R0 is trained against the minimum sufficient profile. R1, R2, and R3A are trained against the minimum-cost factorized action. R2’s 32-dimensional signed-hash input uses the actual question text. All neural controllers use five fixed initialization seeds. The retry audit begins from one frozen R0 seed and permits one single-control correction or a separately labeled compound fallback.

4.3 Inherited measurement scope

Search, gather, page, cache, and batching benchmarks run on CPU with PyTorch 2.12.1. Exact search uses 32 queries, 64-dimensional keys, Top-8, and 256–4,096 memories. Kernel tests use 8 heads and head dimension 32. Page tests store 1,537 tokens and select 128 clustered or dispersed tokens. Batch tests use 1–32 heterogeneous requests. We report medians and mark HBM estimates as extrapolated.

These earlier CPU components and held-out Qwen observations are retained only as provenance for the Paper-5.5 handoff. They are not used to select factorized actions and are not adaptive end-to-end serving measurements.

4.4 Inherited baseline discipline

The matched synthetic corpus contains 1,024 documents and 200 two-hop queries. The query retrieves a bridge; the answer evidence is linked from that bridge. Native context, truncation, one-shot RAG, multi-query RAG, iterative RAG, fixed PRA, and adaptive PRA share the corpus and retrieval geometry. This controlled construction tests cost accounting and the need for iteration. It does not replace a full production RAG evaluation [6, 13].

H2O [17], SnapKV [7], StreamingLLM [15], and ClusterKV [8] are represented by mechanism-faithful controlled selectors at matched active-K/V budgets. No upstream kernel is claimed. Sparse architectural methods are discussed rather than experimentally mixed with incompatible checkpoints.

5 Adaptive-Compute Results

5.1 Oracle ceiling and direct control

The oracle audit is strongly dataset-dependent. On HotpotQA, 31 of 35 held-out units require E2, two require E1, and two require E0. On QASPER, 24 of 30 require only E0 and six require E2. This heterogeneity creates a large QASPER adaptive-compute opportunity but almost none on HotpotQA.

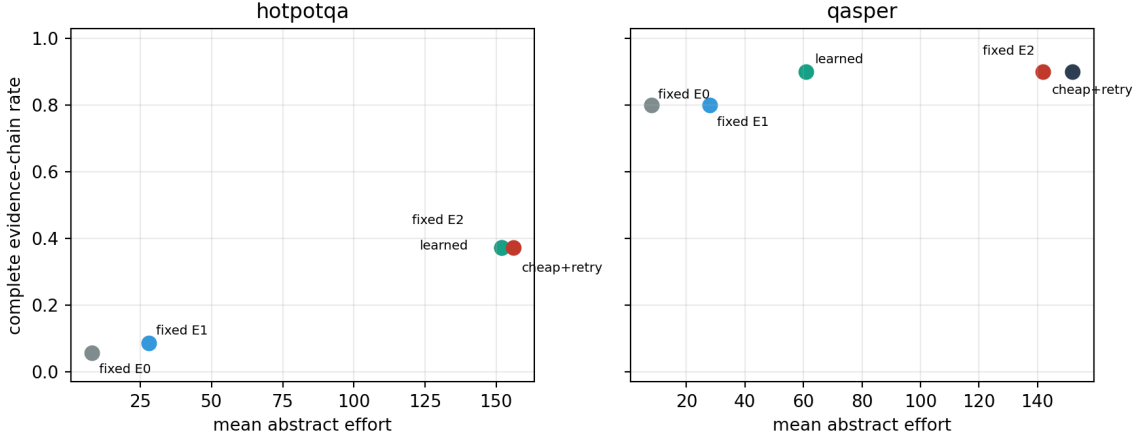


Figure 2: Held-out complete-chain quality versus mean abstract effort. Learned control matches E2 on both datasets. It saves effort on QASPER but not on the predominantly hard HotpotQA cohort.

Table 2: Held-out adaptive-compute frontier. Cost includes conceptual search and physical K/V; retry rows add the controlled regeneration charge.

Dataset	Policy	chain quality	mean cost	attempts	escalation
HotpotQA	fixed E0	.057	8.0	1.00	.000
HotpotQA	fixed E2	.371	152.0	1.00	.000
HotpotQA	learned direct	.371	152.0	1.00	.000
HotpotQA	cheap + retry	.371	156.0	3.00	1.000
QASPER	fixed E0	.800	8.0	1.00	.000
QASPER	fixed E2	.900	152.0	1.00	.000
QASPER	learned direct	.900	60.8	1.00	.000
QASPER	cheap + retry	.900	141.9	2.83	.933

The learned direct policy reduces QASPER mean effort by 60.0% with no chain-quality loss. Its mean oracle cost regret is 24 units and quality match is 1.0, so further gains remain possible. On HotpotQA, the learned and hand controllers both match oracle-profile quality but incur positive regret because a few easy cases are conservatively sent to E2.

5.2 Retry recovery is not free

Cheap-first retry corrects 31.4% of HotpotQA units that fail E0 and 10.0% of QASPER units. It does not break an initially correct unit in this trace. However, HotpotQA reaches E2 on every request and pays two extra generation attempts, raising cost from 152.0 to 156.0. QASPER stops early on two units but still escalates 93.3%, reducing cost only to 141.9. Retry is valuable as a recovery and fallback mechanism; the learned initial profile is the main source of QASPER savings.

This result also distinguishes routing reuse from generation reuse. Roots and K/V states can be preserved incrementally, but a changed answer may still require regeneration. A serving claim must measure discarded tokens and prefix-cache reuse rather than treating monotonic search as free.

5.3 Confidence and selective risk

Table 3: Held-out incorrect/failure prediction. Output rows use the controlled Paper-3 model; routing rows use frozen Paper-2.5 traces.

Cohort	Signals	AUROC	AUPRC	ECE	Brier
Output	entropy only	.403	.899	.229	.145
Output	answer margin only	1.000	1.000	.277	.133
Output	combined output	1.000	1.000	.291	.117
Routing	routing entropy only	.652	.690	.043	.249
Routing	root gaps	.802	.816	.117	.197
Routing	combined runtime	.824	.853	.139	.187

The output cohort is small (40 held-out observations, 90% incorrect), so perfect rank separation for answer margin is diagnostic rather than a deployment estimate. Entropy-only ranking is worse than random in this cohort despite high AUPRC caused by class imbalance. No incorrect output is assigned less than .2 error probability by the fitted predictors, but this does not establish that confidently wrong answers are absent in broader language tasks.

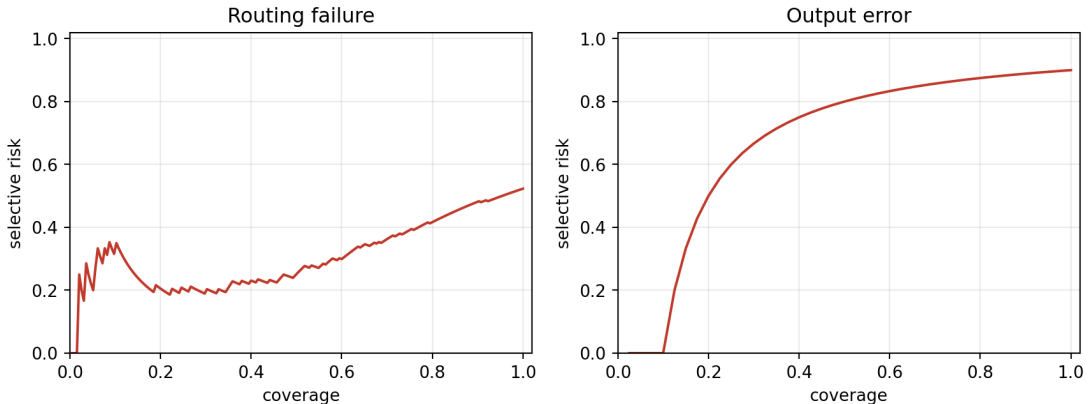


Figure 3: Selective risk as increasingly uncertain observations are accepted. Curves are reported separately because routing and output confidence were not observed on one unified live cohort.

6 Query-Region and Router-Architecture Results

6.1 Matched prompt layouts

The controlled corpus crosses 24 semantic cases, three payload forms (prose, logs, and URL lists), and five serializations. Each row keeps evidence, eight candidate roots, one selected root, 16 active native-K/V tokens, and a global facet fixed. The lexical candidate geometry is intentionally simple: the experiment isolates cue position and role detection, not pretrained semantic routing.

Table 4: Matched query-layout headline result. Each policy cell is root R@1 / answer-quality proxy; active K/V is 16 throughout. Effort lists head / explicit / structural / reinterpretation lexical comparisons.

Layout	head	explicit	structural	head+reinterpret	active K/V	effort
$C + Q$	1.00/1.00	1.00/1.00	1.00/1.00	1.00/1.00	16	8/8/8/8
$Q + C$	.00/.00	1.00/1.00	1.00/1.00	1.00/1.00	16	8/8/8/16
$C_1 + Q + C_2$	.00/.00	1.00/1.00	1.00/1.00	1.00/1.00	16	8/8/8/16
$I + C + Q$	1.00/1.00	1.00/1.00	1.00/1.00	1.00/1.00	16	8/8/8/8
$Q + \text{long payload}$	.00/.00	1.00/1.00	1.00/1.00	1.00/1.00	16	8/8/8/16

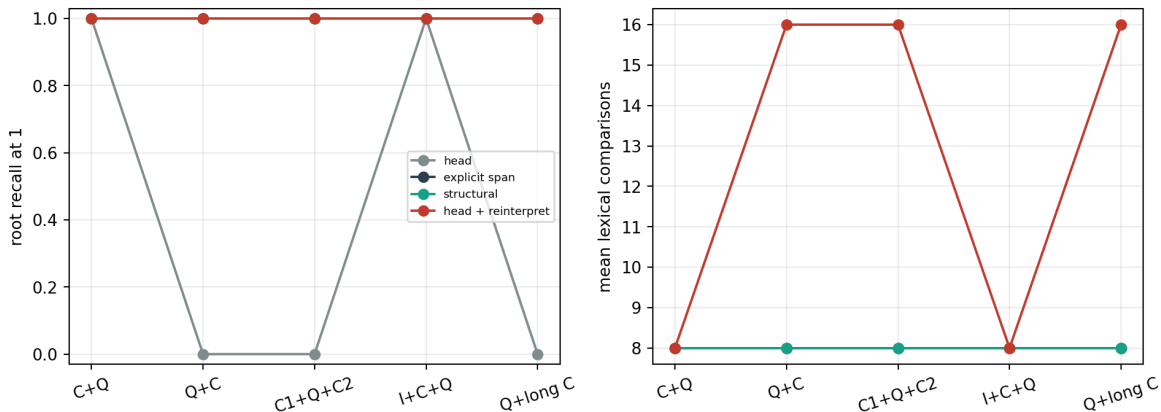


Figure 4: Serialization changes a suffix-head cue but not explicit or structurally discovered regions in the matched control. Reinterpretation pays a second search only when the structural cue does not overlap the recent head.

Head R@1 averages .400 and its layout spread is 1.0. Explicit and structural regions both average 1.000 with zero spread, so Gate 1 and Gate 2 pass in this corpus. Reinterpretation corrects all 216 head failures, breaks none of 144 initially correct cases, and adds eight comparisons per corrected case (4.8 averaged over all requests). This perfect heuristic result follows from explicit question labels and punctuation; it is a structural control, not evidence that natural query-role discovery is solved.

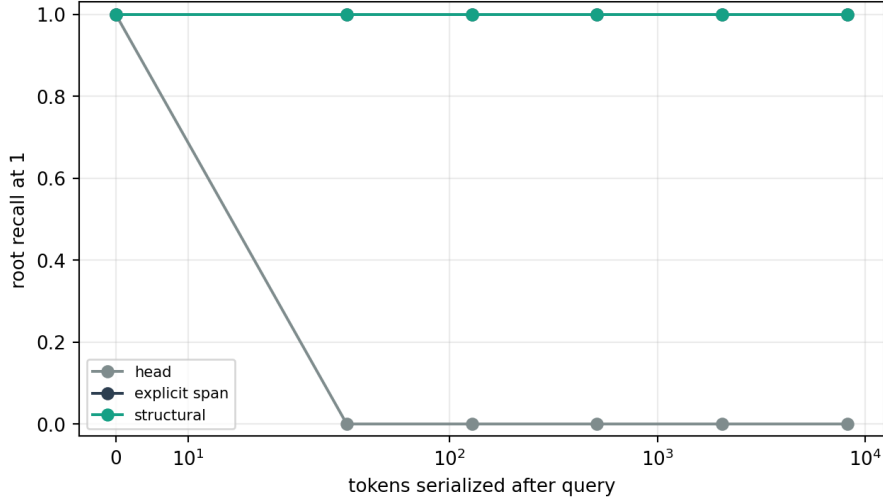


Figure 5: Early-query displacement. The 16-token head fails once payload follows the query, while explicit and structural regions preserve root R@1 through 8,192 following payload tokens.

The displacement sweep makes the motivating failure direct: head R@1 changes from 1.0 at zero following tokens to zero at 32 and remains zero through 8K; explicit and structural selection remain 1.0. Multi-region, facet, root-count, URL-role, and session-isolation rows are retained in the public artifacts. Broad all-region selection reaches only .333 overall, so more query-region breadth is not monotonically better.

6.2 Semi-natural and adversarial role fixtures

Six additional fixtures place the operative question inside email, tool output, source code, logs, URL lists, and multi-turn conversation. Structural discovery obtains root R@1 1.000 and mean span IoU .981; explicit spans obtain 1.000 on both measures. These remain constructed fixtures, but their surface form is less regular than the original matched template.

Five adversarial fixtures insert a quoted previous question, a question-like log field, a misleading heading, a code-comment question, or a URL containing query-like text. Structural R@1 falls to .400 and span IoU to .314. Recent-head remains correct on all five because the current question is later than the decoy. The bounded policy therefore keeps the recent interpretation rather than replacing it merely because a structural cue exists. This negative control changes the design conclusion: structural discovery is one candidate interpretation, and disagreement or a small score gap should trigger comparison, not unconditional trust.

6.3 Preliminary profile-derived router capacity

R0–R3A share the 95 validation / 65 test split and validation-derived minimum-effort labels. R1 uses runtime features only. R2 adds a 32-dimensional signed-hashing query channel to exercise the semantic-router interface without downloading or claiming a pretrained encoder. R3A adds autoregressive parameter conditioning. R1–R3A use five seeds.

Table 5: Held-out router architecture frontier. Quality is complete-chain rate; cost is abstract PRA effort. R0 has one deterministic fit; learned multi-head values pool five seeds.

Router	quality	cost	joint exact	mean head acc.	under	over
R0 profile	.615	109.9	.800	.800	.031	.169
R1 feature MLP	.594	103.7	.775	.779	.055	.166
R2 semantic MLP	.591	104.0	.794	.799	.046	.157
R3A autoregressive	.591	102.6	.806	.806	.049	.145

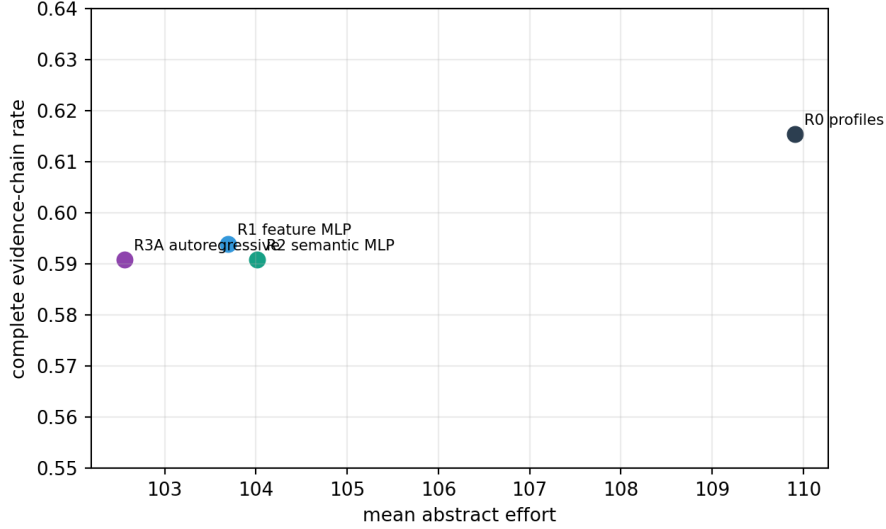


Figure 6: Added controller capacity does not improve the held-out quality/cost frontier. R3A is slightly cheaper than R2 at the same quality, but both remain below R0 quality.

The profile selector remains the supported controller. R1–R3A save 5.9–7.3 cost units relative to R0 by under-allocating more often, but lose 2.2–2.5 quality points. R3A improves joint parameter exactness to .806 and lowers cost relative to R2 without restoring quality. The interaction gate therefore fails: no controller transformer is built. Parameter mutual information is reported only as profile-label coupling because all heads inherit three jointly defined profiles; it is not causal evidence that changing one PRA control changes another.

7 Factorized Control Results

7.1 Measured action lattice and cost

The preliminary multi-head comparison cannot answer whether profiles are the right action unit: every target field was copied from E0, E1, or E2. We therefore replay the frozen native adjacency and query-facet scores over

$$(F, R, K, H, B_s, B_{kv}) \in \{1, 2, 4\} \times \{1, 2, 4\} \times \{2, 4, 8\} \times \{0, 1, 2, 3\} \times \{2, 4, 8\} \times \{2, 4, 8\}. \quad (11)$$

Configurations with $B_s < R$ or $B_{kv} < R$ are invalid, leaving 792 measured actions per example–seed unit. Query-region policy is fixed to structural, native edge admission is open, terminal-goal

stopping is closed, and best-first traversal is fixed. These controls isolate profile coupling rather than reopening every Paper-2.5 routing choice.

For each execution, root scoring compares F facets with all source parents; graph search records actual native proposals; and physical admission preserves roots before admitting discovered parents in search order. The normalized accounting used only for within-study ordering is

$$C(a) = F + N_{\text{transition}} + B_s + \frac{T_{kv}}{64}, \quad (12)$$

where T_{kv} is the number of materialized native-K/V source tokens. The artifacts also retain unscaled root comparisons FN_{parents} , transition comparisons, conceptual parents, K/V tokens, and wall time. Thus Equation 12 is auditable rather than a latency surrogate.

Physical evidence metrics are token weighted:

$$P_{E,kv} = \frac{\text{materialized evidence K/V tokens}}{\text{all materialized K/V tokens}}, \quad R_{E,kv} = \frac{\text{materialized evidence K/V tokens}}{\text{required evidence K/V tokens}}. \quad (13)$$

Complete-chain quality requires every evidence group to be represented after physical admission. No generated answer exists for these newly factorized configurations, so no output oracle is imputed.

7.2 Profile quantization regret

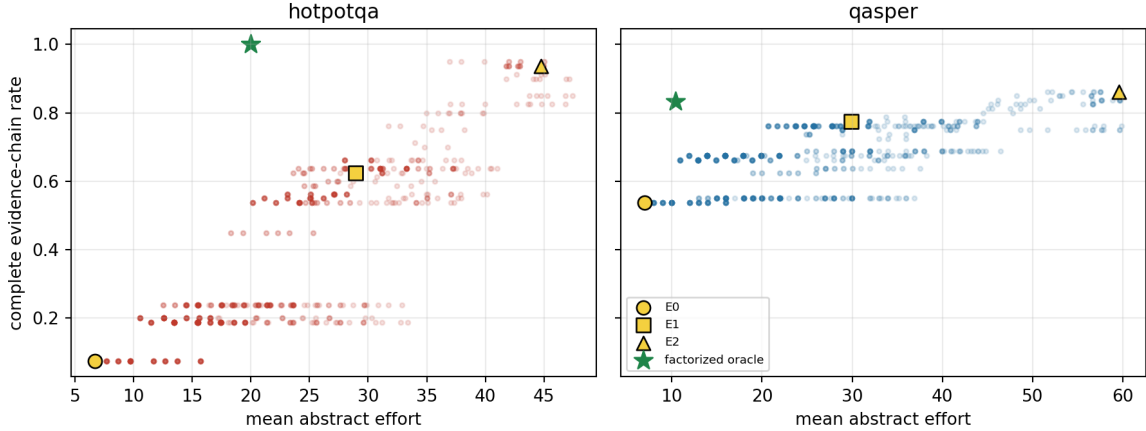


Figure 7: Held-out factorized quality–cost surface. Stars are per-example factorized oracles averaged by dataset; E0–E2 are diagonal profile configurations under the same accounting.

Table 6: Held-out profile quantization audit. Matched rows include units solved by both action spaces. QASPER overall quality is also shown because factorization solves one additional unit.

Dataset	matched n	profile quality	factor quality	profile cost	factor cost	regret
HotpotQA	35	1.000	1.000	34.18	19.98	14.20
QASPER matched	24	1.000	1.000	16.11	8.95	7.15
QASPER all	30	.800	.833	16.82	10.43	—

HotpotQA supports the profile-aliasing hypothesis rather than the intrinsically-high-effort hypothesis: difficult examples often need one expensive dimension, not the full E2 bundle. Mean cost falls

41.5% at matched complete-chain quality. On matched QASPER units the aggregate reduction is 44.4%. The absolute costs should not be compared with the earlier 8/28/152 convention; Table 6 recomputes both spaces with Equation 12.

7.3 What the oracle actually requests

Table 7: Held-out minimum-cost factorized-oracle distributions. Entries are percentages of example-seed units; omitted levels have zero mass.

Control	HotpotQA	QASPER
F	1: 97.1, 4: 2.9	1: 96.7, 2: 3.3
R	1: 65.7, 2: 8.6, 4: 25.7	1: 66.7, 2: 23.3, 4: 10.0
K	2: 80.0, 4: 5.7, 8: 14.3	2: 96.7, 4: 3.3
H	0: 45.7, 1: 54.3	0: 96.7, 1: 3.3
B_s	2: 34.3, 4: 45.7, 8: 20.0	2: 86.7, 4: 10.0, 8: 3.3
B_{kv}	2: 34.3, 4: 45.7, 8: 20.0	2: 86.7, 4: 10.0, 8: 3.3

Nearly every unit uses one facet and low successor breadth. HotpotQA’s extra work is concentrated in roots, one graph hop, and larger parent budgets; QASPER is predominantly root-only. No held-out oracle chooses $B_s > B_{kv}$. Broader-search/narrower-admission configurations are measured, but under deterministic search-order admission they do not become the cheapest complete physical chain. Separating the budgets remains architecturally necessary; this admission rule does not yet exploit it.

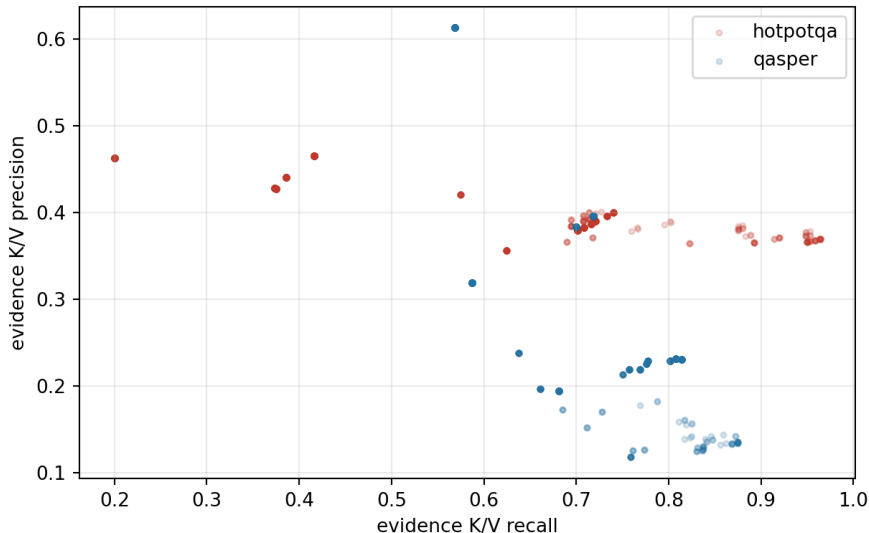


Figure 8: Token-weighted physical evidence precision and recall across aggregate factorized configurations. Broad conceptual recovery does not guarantee a precise admitted K/V set.

7.4 Routers retrained on factorized targets

Every learned router is trained only on 95 validation units and evaluated on 65 test units over five initialization seeds. R2 now hashes actual question text rather than a dataset-class phrase. Conservative R1/R2 decoding selects an upper categorical quantile. Invalid independently predicted budgets are raised only enough to preserve all roots, and the repair rate remains below 2.2%.

Table 8: Five-seed held-out router frontier on factorized targets. Under-allocation means failing a chain the factorized oracle completes; over-allocation means matching quality at higher cost.

Router	quality	cost	cost regret	under	over
R0 profile	.631	24.29	8.72	.292	.523
R1 independent	.538	16.13	.56	.385	.378
R1 conservative	.560	17.97	2.40	.363	.422
R2 query hash	.505	16.55	.98	.418	.335
R2 conservative	.548	18.06	2.48	.375	.400
R3A autoregressive	.486	17.18	1.61	.437	.335

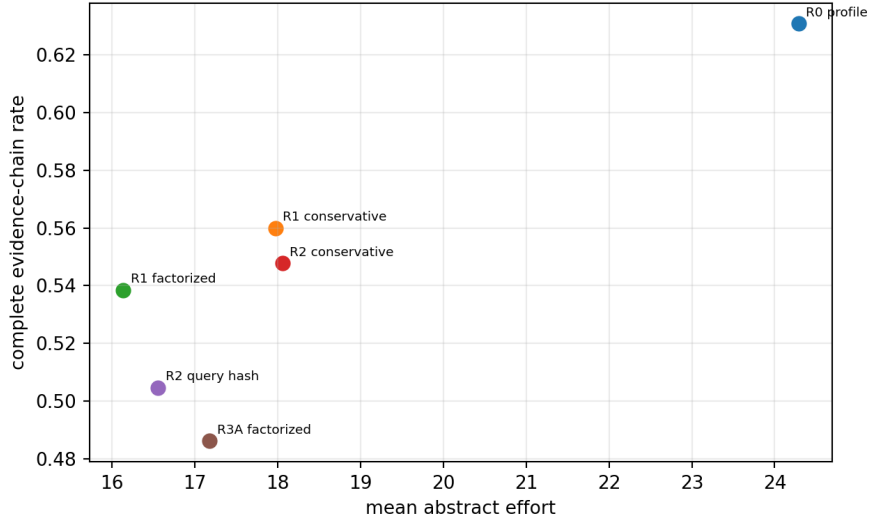


Figure 9: Factorized controller quality versus expected cost. Lower nominal cost is obtained by missing sufficient actions more often; no factorized learned router dominates R0.

True factorized labels prove that cheaper actions exist, but the small cohort and signed-hash channel do not predict them safely. Conservative decoding recovers 2.2–4.3 quality points over its argmax counterpart, still below R0. A pretrained encoder or larger controller is not promoted here; the next gate is a larger grouped natural dataset.

7.5 Bounded targeted-retry headroom

We take one frozen-seed R0 decision as the initial action and permit one corrective attempt. The audit first chooses the cheapest successful action differing in exactly one control. If none exists, it records a compound fallback to the factorized oracle. Accounting preserves prior effort and adds two regeneration units. Because evaluator success chooses the correction, this is an upper bound, not an online classifier.

Table 9: Held-out bounded corrective-retry audit. Parenthetical values give corrected initial failures.

Dataset	initial quality	final quality	corrected	mean added cost	correct broken
HotpotQA	.571	1.000	.429 (15/15)	2.31	0
QASPER	.700	.833	.133 (4/9)	2.71	0
All	.631	.923	.292 (19/24)	2.49	0

Seven Hotpot failures are repaired by increasing R , one by widening F , and seven require a compound fallback. One QASPER failure is repaired by increasing R ; eight invoke compound fallback, of which only three are solvable anywhere in the lattice. The audit establishes retry headroom and localizes missing capacity. It does not establish that an oracle-free diagnostic attains that rate. The endpoint remains direct initial prediction followed by at most one confidence-triggered, explicitly traced correction.

8 Inherited Runtime Measurements and Scope Handoff

The following measurements were completed during the original combined control/serving study and are retained for provenance. They are not Paper 3.5 contributions. Kernel compilation, memory hierarchy, cache/prefetch policy, batching, concurrency, and broad engine comparison now belong to Paper 5.5; first-class vLLM logical blocks and scheduling belong to Paper 6.

8.1 Exact GEMM is the current small-index default

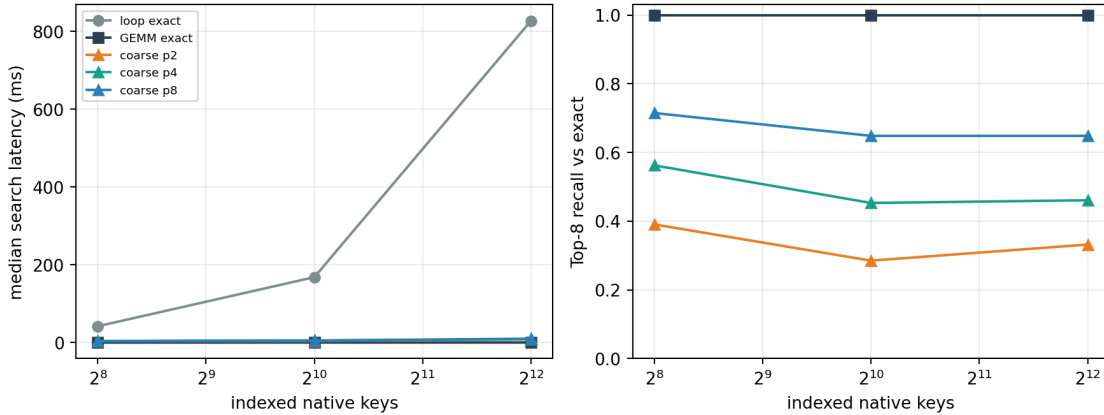


Figure 10: Native-Q/K search. Exact GEMM removes Python per-key overhead. The present deterministic coarse index loses recall and does not yet beat GEMM at these memory sizes.

At 4,096 keys, the Python loop takes .827 seconds for 32 queries while exact GEMM takes .327 ms, a 2,527 $\times$ speedup with identical Top-8 results. Coarse-to-fine search with two, four, and eight probes reaches .332, .461, and .648 recall relative to exact, respectively, while taking 8–13 ms. The ANN path is therefore not adopted as a default. It requires a mature FAISS/HNSW/GPU index and larger-scale crossover measurements before it can support a speed claim.

8.2 Gather and page layout

Table 10: Median standalone CPU gather time in milliseconds. Every row has exact K/V parity.

Source tokens	Python slice/cat	fused index-select
512	.197	.182
2,048	.195	.227
8,192	.191	.198

The one-index path has exact parity but no consistent CPU speed advantage in this small benchmark; it is not a fused CUDA kernel. The fixed-page Python lookup is substantially slower: depending on page size and selection dispersion, it costs 14–37 $\times$ contiguous `index.select`. Pages of 16–128 tokens waste 15–127 tail slots for the 1,537-token request. Clustered selection touches 9, 5, 3, and 2 pages; dispersed selection touches 97, 49, 25, and 13. These numbers explain what a production page gather should optimize, not how fast that gather will be.

8.3 Cache and prefetch

Without prefetch, the best controlled hit rate is .921 at 32 pages; LRU, LFU, and hybrid tie because the full working set nearly fits. At 16 pages LFU reaches .896, above LRU’s .833, because the workload contains a stable hot set plus a moving path. Graph-neighbor prefetch is measured, while the next-access trace is reported only as an oracle upper bound and never as a deployable predictor. Useful and wasted prefetch transfers are explicit artifact columns.

8.4 Heterogeneous batching

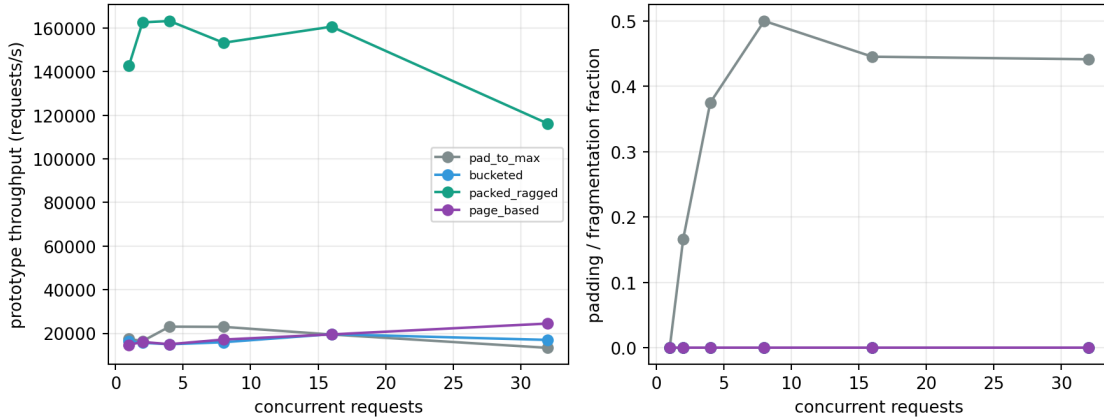


Figure 11: Controlled CPU batching. Packed ragged execution avoids padding and is fastest for this small per-request matrix-vector workload; this is not a GPU attention-kernel benchmark.

At concurrency 32, pad-to-maximum wastes 44.1% of allocated token slots. Bucketed, page-based, and packed layouts have zero waste for the chosen lengths. Packed per-request operations reach 128,154 requests/s versus 13,273 for pad-to-maximum in this CPU microbenchmark. On GPU, launch overhead and fused kernels can reverse this ordering; the artifact therefore records device and measurement scope.

8.5 Inherited Qwen serving observations

Table 11: Mean held-out single-request Qwen observations inherited from Gate 3. Active K/V counts layer-token states, not unique source tokens.

Dataset	Effort	active K/V states	TTFT (s)	total (s)
2Wiki	E0	2,200	.169	1.000
2Wiki	E1	4,267	.190	1.348
2Wiki	E2	6,215	.198	1.465
MuSiQue	E0	7,562	.320	3.740
MuSiQue	E1	44,683	.705	7.688
MuSiQue	E2	47,869	.737	7.936

These rows demonstrate that effort can materially change K/V and latency. They do not demonstrate that the new controller produces the same values end to end, because the adaptive controller was evaluated on frozen routing traces rather than a live Qwen scheduler. GPU utilization and dollar cost were not captured on the local hardware and remain explicitly missing.

9 Matched Baselines

9.1 RAG, iterative retrieval, and long context

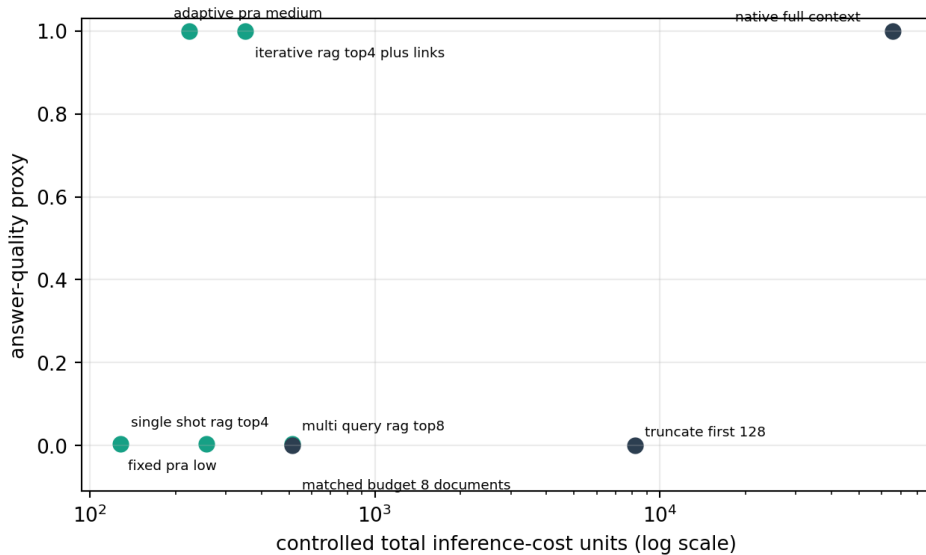


Figure 12: Controlled two-hop quality/cost frontier. Iterative RAG is the appropriate comparison for iterative PRA; a one-shot RAG baseline is insufficient.

Table 12: Controlled matched-corpus retrieval results. Quality is target-evidence recovery.

Method	quality	active K/V	cost units
one-shot RAG Top-4	.005	256	256
multi-query RAG Top-8	.005	512	512
iterative RAG Top-4 + links	1.000	320	352
fixed PRA low	.005	128	128
adaptive PRA medium	1.000	192	224
native full context	1.000	65,536	65,536

The corpus is constructed so the first retrieval identifies a bridge and the answer lies behind its link. One-shot methods fail by design; iterative RAG and adaptive PRA recover the target. PRA uses fewer controlled active tokens because it activates native memory rather than serializing all selected documents into a prompt. This supports the accounting distinction, not a general claim that PRA dominates optimized iterative RAG.

9.2 K/V selection proxies

At budget 32, the H2O, SnapKV, StreamingLLM, ClusterKV, and native-Q/K proxies recover the planted salient token with rates .915, 1.000, .120, .045, and 1.000. At budget 128 the rates are 1.000, 1.000, .240, .255, and 1.000. These outcomes reflect the controlled score construction. They test matched-budget bookkeeping and expose the expected failure of recent-only selection when evidence is not recent. They do not rank the upstream systems on real language generation.

Dense and external retrieval solve different physical problems. H2O and SnapKV select or compress an existing sequence cache; RAG retrieves text for prompt ingestion; PRA selects addressable contextual K/V. Longformer [2], BigBird [16], Reformer [4], and Routing Transformer [12] change the attention architecture and require compatible checkpoints. Compressive [11], memorizing [14], Infini-Attention [10], Landmark Attention [9], RETRO [3], and Titans [1] use recurrent, compressed, or external memory. We preserve this taxonomy rather than forcing unfair experimental rows.

10 SDK and Reimplementation Contract

The runtime is exposed through Python data classes and the `pra-hf` `adaptive` CLI. A caller can inspect profiles or resolve an automatic plan:

```
pra-hf adaptive profiles --output adaptive_effort_profiles.json
pra-hf adaptive plan --mode auto --features request_features.json \
  --controller controller.json --max-retries 2 \
  --max-search-budget 8 --max-active-kv 1024 \
  --latency-budget 1.5 --confidence-threshold 0.35 \
  --trace-output request_plan.json
```

Query intent can be supplied as a token span, structured roles, or inferred structurally:

```
selector.select(prompt, query_spans=((q_start, q_end),))
selector.select(segments=[
    PromptSegment("query", question),
    PromptSegment("context", logs),
])
selector.select(prompt, policy="structural")
```

The selector returns half-open token intervals, confidence, score margin, method, and selected text. The model adapter can use those intervals to construct facets without assuming that query end equals prompt end. Structured segments are preferred in public applications; automatic inference does not override explicit metadata.

Manual mode uses the same profile representation and accepts explicit query-region, facet, root, neighbor, hop, conceptual-budget, K/V, threshold, layer, granularity, and materialization overrides. Overrides are request-local; they do not silently redefine the validated global ladder.

An independent implementation needs four invariants:

1. Separate conceptual and physical budgets in the public request and trace schema.
2. Reject evaluator labels before feature vectorization; do not merely promise not to use them.
3. Preserve prior selected identities and pages when escalating, and account for regenerated tokens separately.
4. Label every timing as component, end-to-end, measured, or extrapolated. A Python microbenchmark cannot become a production throughput claim by omission.
5. Keep query-role discovery separate from faceting, and preserve explicit caller spans or roles through retries unless the caller permits reinterpretation.

The relevant implementation is compact. `adaptive_runtime.py` defines profiles, feature validation, learned and hand controllers, stop policy, retries, consistency, and calibration. `query_regions.py` defines structured segments, explicit spans, structural discovery, multi-region selection, and cue reinterpretation. `effort_router.py` defines finite action spaces, independent categorical heads, a cached semantic-input contract, and autoregressive heads. `factorized_control.py` defines valid independent actions, exact component accounting, token-weighted evidence metrics, Pareto dominance, allocation outcomes, and retry action names. `factorized_study.py` executes the frozen lattice and emits the oracle, router, and retry artifacts. Physical-runtime implementation remains behind the control/execution boundary.

11 Discussion

11.1 What adaptive control accomplished

The profile experiment first appeared to make HotpotQA uniformly high effort: 31 of 35 held-out units needed E2. Factorization changes that interpretation. Every Hotpot unit is solved at lower mean cost because most need only one facet, low K , and at most one hop; the expensive requirement is usually concentrated in roots or total parent budget. Coherent profiles are robust, but they pay for controls that an individual query does not require.

QASPER remains the more heterogeneous workload. Its factorized oracle uses $H = 0$ on 96.7% of held-out units and obtains a small quality gain over the profile lattice. These oracle results justify fine-grained actions in principle. The learned-router results deny the stronger claim that the present controller can allocate them safely: every R1–R3A variant loses quality through under-allocation. R0 therefore remains the supported initial controller despite measurable quantization regret.

Retry is similarly conditional. A bounded evaluator-side correction closes 19 of 24 initial failures, and eight corrections change only roots or facets. The remaining solvable failures need compound

changes. This establishes a target for a future diagnostic classifier, not an achieved online correction policy. Universal cheap-first escalation remains inferior to direct prediction.

The add-on studies sharpen what escalation means. Failure may require a different cue rather than more roots under the same misplaced cue. Explicit roles and spans are both cheaper and more reliable than discovering role from raw serialization, so applications should expose them when possible. Automatic structural reinterpretation is a bounded fallback. Query-region breadth has the same two-sided character as other PRA controls: it can recover distributed instructions, but it can also dilute the objective and multiply facets, roots, and comparisons.

They also provide a controller-complexity stop. Independent and autoregressive heads now receive genuine factorized targets, yet neither improves the quality/cost frontier. The result is no longer an artifact of copying E0/E1/E2 labels, although the sample and query representation remain limited. A compact pretrained encoder should be tested only on a larger grouped natural cohort.

11.2 Cognitive interpretation

The tested computational principle is modest:

cheap or predicted effort $\rightarrow$ uncertainty detection $\rightarrow$ selective escalation $\rightarrow$ stop.

Natural cognitive systems plausibly vary retrieval cues and deliberation under uncertainty, but the present controller is not a biological model. Its scientific content is the measured frontier between quality and expected resource use.

11.3 Program handoff

Paper 3.5 ends at an adaptive initial action, observable sufficiency evidence, and one bounded correction. Kernel compilation, indexed-search crossover, physical memory hierarchy, batching, and concurrency are Paper 5.5 questions. First-class vLLM logical blocks, asynchronous memory movement, and scheduler awareness are Paper 6 questions. This boundary keeps control quality from being conflated with a Python prototype’s throughput.

12 Limitations

The adaptive quality endpoint is evidence-chain completion from frozen traces, not a live generated answer. Output confidence comes from a separate controlled model, so a combined routing/output controller remains untested. The held-out adaptive cohort contains only 65 example-seed units and inherits Paper-2.5’s model and dataset choices.

The supported learned controller is the R0 profile selector. Factorized R1–R3A use independently measured targets, but the cohort is too small to establish reliable natural parameter interactions. R2 uses actual questions through a dependency-free signed-hash encoder rather than a pretrained NLP encoder. The output calibration cohort is small and imbalanced. Semantic consistency uses token F1 and can miss paraphrase and contradiction.

Query-region evaluation remains deterministic. Semi-natural email, code, log, URL, tool-output, and conversation fixtures broaden surface form but are not sampled from a natural prompt distribution. The adversarial structural failure shows that ambiguous role resolution remains open. Tokenizer- specific span mapping and a lightweight learned span scorer remain untested.

The factorized endpoint is frozen-score evidence-chain recovery, not generated-answer quality. Facet selection ranks existing cached facets; it is not a newly learned interpreter. Search-order

admission does not exploit $B_s > B_{kv}$ in any held-out minimum-cost oracle. The targeted correction audit uses evaluator success to choose an action and must not be read as an oracle-free retry result. Five QASPER units remain unsolved throughout the measured lattice.

Finally, no full PRA backbone is trained here. Learned consumer adaptation and full-weight scaling are deferred to Paper 4.

13 Conclusion

PRA effort is better represented by separable interpretation, search, and admission controls than by one diagonal profile lattice. At matched complete-chain quality, factorization reduces mean held-out HotpotQA cost from 34.18 to 19.98 and matched QASPER cost from 16.11 to 8.95. The result resolves the main scientific question: profile quantization hides useful low-cost combinations.

The control problem is not solved. R1–R3A reduce nominal cost by under-allocating and remain below R0 quality, even with factorized targets and actual-question hashing. R0 stays the SDK default. One bounded oracle corrective action exposes substantial retry headroom, but compound changes and five unsolved QASPER units prevent a stronger online claim.

Adaptive PRA must also decide what the query is. Explicit roles remain preferred; structural and recent interpretations should compete under confidence because either can fail under serialization or decoys. The contribution is therefore a measured control surface, a conservative default, and a bounded retry endpoint. Making those decisions fast belongs to Paper 5.5; making them native to vLLM belongs to Paper 6.

A Artifact Manifest

All normalized artifacts are under `docs/papers/shared/results/paper3_5_adaptive_pra`. They include:

- effort profiles, controller features/training/calibration, oracle ceiling, regret, retry traces/outcomes, and adaptive frontier;
- query-region configurations plus per-layout, detection, retrieval, output, displacement, facet, root, retry, session, semi-natural, and adversarial summaries, findings, and figures;
- router profiles, action spaces, oracle targets, R0–R3 results, per-head calibration, interaction diagnostics, regret, latency, query-class summaries, findings, and frontier figure;
- factorized action space, per-unit oracles and Pareto fronts, quantization regret, parameter requirements/interactions, precision–recall and search–admission surfaces, factorized router targets/results, bounded retry rows, findings JSON, and figures;
- inherited systems and matched-proxy rows retained for Paper-5.5 provenance, not counted as Paper-3.5 contributions.

The complete study runs with:

```
python -m experiments.paper3_5_adaptive_pra.run_study
python -m experiments.paper3_5_adaptive_pra.factorized_study
python -m pytest
```

B Decision and Accounting Details

The learned controller uses standardized ridge least squares against one-hot E0/E1/E2 targets. The failure controller uses the same estimator with success/failure targets. Validation selects a .3 maximum predicted-failure threshold for retry stopping. AUROC is computed by exact pairwise ranking, AUPRC by average precision over descending risk, ECE with ten equal-width bins, and Brier score as mean squared probability error.

Risk-coverage sorts examples from lowest to highest predicted failure. At coverage c , selective risk is the failure fraction among the lowest-risk cN observations. We retain every point in the calibration JSON rather than reading values back from a rendered figure.

Search comparisons count query-key dot products. K/V bytes count both keys and values. Page fragmentation is allocated page slots minus valid tokens. Batch allocation counts padded or page rounded token slots. CPU benchmark bytes are physical tensor sizes; HBM/request is an explicitly marked extrapolation from token count, dimension, dtype, and K/V multiplicity.

For factorized control, profile quantization regret is defined only when both the profile and factorized action spaces contain a complete-chain configuration:

$$R_{\text{quantization}} = C_{\text{profile oracle}} - C_{\text{factorized oracle}}. \quad (14)$$

Under-allocation means the selected action misses a chain recovered by the factorized oracle. Over-allocation means it matches oracle quality at higher cost. A failed unit is never counted as a cost saving. Held-out oracle labels are used only for evaluation and corrective-action upper bounds.

C Representative Public Trace

```
{
  "attempt": 1,
  "effort": "E1_medium",
  "control_vector": {
    "F": {"policy": "multi_span", "count": 2},
    "R": 2, "K": 4, "H": 1,
    "B": {"conceptual_parents": 4, "native_kv_tokens": 512},
    "theta": 0.45,
    "L": {"search": [24, 27], "consumer": [26, 27]},
    "G": 128, "M": "evidence_centered"
  },
  "active_native_kv": 512,
  "incorrect_probability": 0.28,
  "retry_consistency": 0.83,
  "reused_search_items": 1,
  "reused_kv_tokens": 256,
  "stop_reason": "confidence"
}
```

This trace is a runtime decision record. It gives an application enough information to display effort and cost or request a manual retry, without exposing hidden reasoning.

References

- [1] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2025.

- [2] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- [3] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George B. M. van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning*, 2022.
- [4] Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *International Conference on Learning Representations*, 2020.
- [5] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*, 2023. Preprint: <https://arxiv.org/abs/2309.06180>.
- [6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.
- [7] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, 2024. Preprint: <https://arxiv.org/abs/2404.14469>.
- [8] Guangda Liu, Chengwei Li, Jieru Zhao, Chenqi Zhang, and Minyi Guo. ClusterKV: Manipulating LLM KV cache in semantic space for recallable compression. *arXiv preprint arXiv:2412.03213*, 2024.
- [9] Amirkeivan Mohtashami and Martin Jaggi. Landmark attention: Random-access infinite context length for transformers. *arXiv preprint arXiv:2305.16300*, 2023.
- [10] Tsendsuren Munkhdalai, Manaal Faruqui, and Siddharth Gopal. Leave no context behind: Efficient infinite context transformers with infini-attention. *arXiv preprint arXiv:2404.07143*, 2024.
- [11] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*, 2020.
- [12] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics*, 9:53–68, 2021. Preprint: <https://arxiv.org/abs/2003.05997>.
- [13] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 10014–10037, 2023.
- [14] Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations*, 2022.
- [15] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. StreamingLLM: Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.
- [16] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020.
- [17] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2023. OpenReview version: <https://openreview.net/forum?id=RkRrPp7GK0>.